APACHE AIRFLOW TRAINING

Hands-On Lab Guide

Astro CLI — Advanced Operations

Deploy · Trigger · Inspect Logs · Debug
Secrets · Connections · RBAC

Environment	Level	Duration	Airflow	Continuation of
AWS EC2 Ubuntu + Docker	Intermediate	~120 minutes	2.x	Lab Sections 1–8

9 DEPLOY DAG TO ASTRO CLOUD

In previous sections you ran Airflow locally using **astro dev start**. This section covers the full lifecycle of deploying your DAGs to **Astronomer Cloud** — the production-grade managed Airflow platform. The workflow is: authenticate → select deployment → push code.

9.1 Authenticate with Astronomer Cloud

Step 1 — Log in to Astronomer Cloud from the CLI

```
$ astro login
```

✦ **NOTE** A browser window opens at cloud.astronomer.io. Complete the SSO or email/password flow. The token is stored in ~/.astro/config.yaml and auto-refreshed.

```
# Confirm you are authenticated
$ astro workspace list
```

Expected output: a table of workspaces your account has access to.

9.2 List and Select a Deployment

Step 2 — List all deployments in your workspace

```
$ astro deployment list
```

Note the **Deployment ID** (format: *clxxxxxxxxxxxxxxx*) of the target deployment. You will use this in every deploy command.

9.3 Deploy Your DAGs

Step 3 — Deploy the entire Astro project (DAGs + image) to the cloud

```
$ astro deploy <DEPLOYMENT_ID>
```

```
# Example:
$ astro deploy clv3abc123def456
```

Flag	Purpose
--dags	Deploy only DAG files (no image rebuild). Fast, safe for DAG-only changes.
--wait	Wait for deploy to complete before returning. Useful in CI pipelines.
--no-prompt	Skip all confirmation prompts (for automation).
--description	Attach a deploy description visible in the Astronomer UI.

Step 4 — DAG-only deploy (fastest path for DAG changes)

```
# Only push DAG files — no Docker rebuild required
$ astro deploy <DEPLOYMENT_ID> --dags
```

✔ **TIP** Use --dags whenever you only edited .py files in dags/. A full image deploy is only needed when you change requirements.txt, Dockerfile, or plugins/.

9.4 CI/CD Pipeline Integration

Step 5 — Automate deployment via GitHub Actions (example)

```
# .github/workflows/deploy.yml
```

```
name: Deploy to Astronomer
on:
push:
branches: [main]
jobs:
deploy:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: astronomer/deploy-action@v0.4
with:
deployment-id: ${{ secrets.ASTRO_DEPLOYMENT_ID }}
astronomer-key-id: ${{ secrets.ASTRO_KEY_ID }}
astronomer-key-secret: ${{ secrets.ASTRO_KEY_SECRET }}
```

■ **WARNING** Never hard-code API key IDs or secrets in workflow files. Always use GitHub Secrets or a vault.

10 TRIGGER DAG RUNS

Apache Airflow supports multiple trigger mechanisms: schedule-based, manual UI trigger, CLI trigger, REST API trigger, and dataset-based event-driven triggers. This section covers all common methods used in production and training environments.

10.1 Trigger via Airflow UI

Step 1 — Manual trigger from the DAGs list page

Navigate to **http://<EC2_IP>:8080** → log in → locate the DAG → click the ■ **Trigger DAG** button on the right side of the row → optionally provide a JSON config → click **Trigger**.

Step 2 — Trigger with configuration payload (UI)

In the Trigger DAG dialog, paste a JSON object in the **Configuration JSON** box:

```
{
"run_date": "2024-06-01",
"environment": "staging",
"batch_size": 1000
}
```

✦ **NOTE** Access the config inside your DAG using: kwargs['dag_run'].conf.get('run_date')

10.2 Trigger via Astro CLI

Step 3 — Trigger a DAG run using the Airflow CLI (inside the container)

```
# Trigger immediately
$ astro dev run dags trigger my_first_astro_dag
```

```
# Trigger with a specific execution date
$ astro dev run dags trigger my_first_astro_dag \
--exec-date "2024-06-01T00:00:00+00:00"
```

```
# Trigger with config JSON
$ astro dev run dags trigger my_first_astro_dag \
--conf '{"env": "prod", "batch": 500}'
```

10.3 Trigger via Airflow REST API

Step 4 — Enable and call the REST API endpoint

```
# Trigger a DAG run via HTTP POST
$ curl -X POST \
http://<EC2_IP>:8080/api/v1/dags/my_first_astro_dag/dagRuns \
-H "Content-Type: application/json" \
-u "admin:admin" \
-d '{
"dag_run_id": "manual_run_001",
"conf": {"source": "api", "priority": "high"}
}'
```

Expected response (HTTP 200):

```
{
```

```
"dag_run_id": "manual_run_001",
"dag_id": "my_first_astro_dag",
"state": "queued",
"execution_date": "2024-06-01T10:00:00+00:00"
}
```

10.4 Dataset-Triggered DAGs (Airflow 2.4+)

Step 5 — Create a producer and consumer DAG pair

```
# Producer DAG — emits a Dataset on completion
from airflow.datasets import Dataset
MY_DATASET = Dataset('s3://my-bucket/processed/data.csv')

@dag(schedule='@daily')
def producer_dag():
@task(outlets=[MY_DATASET])
def write_data():
pass # write your data here
write_data()

# Consumer DAG — runs when MY_DATASET is updated
@dag(schedule=[MY_DATASET])
def consumer_dag():
@task()
def read_data():
pass # process the dataset
read_data()
```

✔ **TIP** Dataset-triggered pipelines eliminate polling-based sensors. Use them to build event-driven architectures where downstream DAGs only run when upstream data is actually ready.

11 INSPECT LOGS

Airflow generates logs at multiple levels: task instance logs, scheduler logs, webserver logs, triggerer logs, and worker logs. Knowing where to look and how to filter dramatically reduces debugging time.

11.1 View Logs in the Airflow UI

Step 1 — Access task instance logs via Grid View

Navigate to the DAG → click the DAG name → Grid View → click a colored task square → in the popup click **Logs**. Use the **attempt** selector to view logs from previous retry attempts.

Step 2 — Filter log levels in the UI

At the top of the log panel, use the **Log Level** dropdown to filter by DEBUG, INFO, WARNING, or ERROR. For noisy DAGs, start with WARNING to surface only actionable messages quickly.

11.2 View Logs via Astro CLI

Step 3 — Stream all container logs

```
# All containers (webserver + scheduler + triggerer)
$ astro dev logs

# Scheduler only
$ astro dev logs --scheduler

# Webserver only
$ astro dev logs --webserver

# Follow (live stream like tail -f)
$ astro dev logs --scheduler --follow
```

11.3 Access Logs Inside the Container

Step 4 — Shell into the scheduler and read log files directly

```
$ astro dev bash

# Inside the container:
# Task logs are in /usr/local/airflow/logs/
(container) $ ls /usr/local/airflow/logs/
(container) $ ls /usr/local/airflow/logs/dag_id=my_first_astro_dag/

# Find logs for a specific run
(container) $ find /usr/local/airflow/logs -name '*.log' | head -20

# Tail the most recent task log
(container) $ tail -50 /usr/local/airflow/logs/dag_id=my_first_astro_dag/
run_id=manual__2024-01-15T10:00:00/task_id=extract/attempt=1.log
```

11.4 Log Configuration Best Practices

Practice	Detail
Log remote storage	Set AIRFLOW__LOGGING__REMOTE_LOGGING=True + S3/GCS URI for persistent logs across restarts.

Log retention	Set AIRFLOW__LOG_RETENTION_DAYS=30 to auto-clean old logs and prevent disk exhaustion.
Structured logging	Use logging.getLogger(__name__) inside tasks for named, filterable log streams.
Log level tuning	Set AIRFLOW__LOGGING__LOGGING_LEVEL=WARNING in prod to reduce noise; DEBUG only for dev.
Sensitive data masking	Use AIRFLOW__LOGGING__SENSITIVE_VAR_CONN_NAMES to mask secret values in logs.

■ **WARNING** Task logs are stored inside Docker volumes by default. They are lost when you run 'astro dev kill'. Configure remote logging (S3) for any environment you care about.

12 DEBUG DEPLOYMENT ERRORS

Deployment errors in Astro fall into four categories: image build failures, DAG import errors, connection/dependency errors, and runtime task failures. This section provides a systematic debugging methodology for each.

12.1 Image Build Failures

Step 1 — Check the build output for package installation errors

```
# Verbose build output
$ astro dev start --no-cache 2>&1 | tee build.log
$ grep -i 'error\|failed\|could not' build.log
```

Common causes of image build failures:

Error Type	Fix
Package version conflict	Pin compatible versions in requirements.txt. Check Apache Airflow constraints for your version.
Missing system dependency	Add RUN apt-get install -y to your Dockerfile before pip install.
Private PyPI index	Add --extra-index-url https://... to requirements.txt or set in Dockerfile ENV.
Network timeout	Retry with --wait 15m. Behind a proxy? Set HTTP_PROXY in Dockerfile.

12.2 DAG Import Errors

Step 2 — Find all DAGs with import errors

```
# List all DAGs that failed to import
$ astro dev run dags list-import-errors

# Parse a specific file
$ astro dev bash
(container) $ python dags/my_dag.py
(container) $ exit
```

Step 3 — Run DAG integrity tests

```
# Built-in test suite: checks imports, cycles, and tags
$ astro dev pytest tests/dags/test_dag_integrity.py -v

# Run a specific test
$ astro dev pytest tests/dags/ -k 'test_dag_integrity' -v
```

✕ **ERROR** A DAG with an import error is silently excluded from the scheduler. Always run 'dags list-import-errors' after each code change before triggering a run.

12.3 Runtime Task Failures

Step 4 — Identify and re-run failed tasks without re-running the entire DAG

```
# Clear only the failed task instance (enables retry from that step)
$ astro dev run tasks clear my_first_astro_dag transform \
--start-date "2024-01-15" --end-date "2024-01-16"

# Test a single task in isolation (no XCom, no dependencies)
```

```
$ astro dev run tasks test my_first_astro_dag extract 2024-01-15
```

Step 5 — Debug with a Python interactive session inside the container

```
$ astro dev bash
(container) $ python
>>> from airflow.models import DagBag
>>> db = DagBag('/usr/local/airflow/dags', include_examples=False)
>>> print(db.import_errors) # {} means all DAGs loaded OK
>>> dag = db.get_dag('my_first_astro_dag')
>>> dag.test() # runs all tasks in sequence locally
```

✔ **TIP** dag.test() executes every task in the DAG sequentially without the scheduler, XCom persistence, or retries. It is the fastest way to verify end-to-end logic locally.

13 CONFIGURE SECRETS

Hardcoding credentials in DAG files or connections is a critical security risk. Airflow provides a **Secrets Backend** abstraction that allows DAGs to fetch secrets from external vaults at runtime. Astronomer Cloud integrates with HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager, and Azure Key Vault.

13.1 Local Development Secrets (.env)

Step 1 — Store local secrets in the .env file

```
# .env — never commit this file to git!
AIRFLOW_CONN_MY_POSTGRES=postgresql://user:pass@host:5432/db
AIRFLOW_VAR_API_KEY=sk-abc123def456
AIRFLOW_VAR_ENVIRONMENT=development
```

Airflow auto-discovers connections and variables from environment variables that match the naming convention above.

> ✦ **NOTE** AIRFLOW_CONN_ sets a connection. AIRFLOW_VAR_ sets a variable. Both are injected at container start by astro dev start.

13.2 AWS Secrets Manager Backend

Step 2 — Install the AWS provider and configure the backend

```
# requirements.txt
apache-airflow-providers-amazon==8.10.0
```

```
# .env — point Airflow at Secrets Manager
AIRFLOW__SECRETS__BACKEND=airflow.providers.amazon.aws.secrets.secrets_manager.SecretsManagerBackend
AIRFLOW__SECRETS__BACKEND_KWARGS={"connections_prefix": "airflow/connections",
"variables_prefix": "airflow/variables", "region_name": "ap-south-1"}
```

Step 3 — Create a secret in AWS Secrets Manager

```
# Create a connection secret
$ aws secretsmanager create-secret \
--name airflow/connections/my_postgres_conn \
--secret-string '{"conn_type": "postgres", "host": "db.example.com",
"login": "airflow", "password": "s3cur3!", "port": 5432, "schema": "prod"}'
```

```
# Create a variable secret
$ aws secretsmanager create-secret \
--name airflow/variables/api_key \
--secret-string 'sk-prod-abc123def456xyz'
```

13.3 Access Secrets in DAG Code

Step 4 — Retrieve a Variable or Connection inside a task

```
from airflow.models import Variable
from airflow.hooks.base import BaseHook

@task()
def call_api():
```

```
# Fetches from secrets backend transparently
api_key = Variable.get('api_key')
conn = BaseHook.get_connection('my_postgres_conn')
print(f'Host: {conn.host}, Schema: {conn.schema}')
# api_key and conn.password never appear in logs
```

■ **WARNING** Never use print(api_key) or logging.info(password) in task code. Airflow masks known secret keys in logs, but only if AIRFLOW__LOGGING__SENSITIVE_VAR_CONN_NAMES is configured.

14 SETUP SECURE CONNECTIONS

Airflow Connections store credentials for external systems: databases, APIs, cloud services, and message brokers. This section covers creating, testing, and managing connections securely via the UI, CLI, code, and airflow_settings.yaml.

14.1 Create Connections via the Airflow UI

Step 1 — Navigate to Admin → Connections → + Add a new record

Fill in: **Connection ID** (unique key), **Connection Type** (postgres, http, aws, S3, etc.), **Host**, **Schema**, **Login**, **Password**, **Port**. Click **Test** to verify before saving.

14.2 Create Connections via CLI

Step 2 — Use the Airflow CLI to add connections programmatically

```
# Add a PostgreSQL connection
$ astro dev run connections add my_postgres_conn \
--conn-type postgres \
--conn-host rds.ap-south-1.amazonaws.com \
--conn-schema airflow_db \
--conn-login airflow \
--conn-password 'S3cure!Pass' \
--conn-port 5432

# Add an HTTP connection (for REST APIs)
$ astro dev run connections add openai_api \
--conn-type http \
--conn-host https://api.openai.com \
--conn-extra '{"Authorization": "Bearer sk-xyz"}'

# List all connections
$ astro dev run connections list

# Delete a connection
$ astro dev run connections delete my_postgres_conn
```

14.3 Test Connections

Step 3 — Test connectivity from inside the container

```
$ astro dev bash

# Test the PostgreSQL connection using an Airflow hook
(container) $ python -c "
from airflow.hooks.base import BaseHook
conn = BaseHook.get_connection('my_postgres_conn')
hook = conn.get_hook()
hook.test_connection()
"
```

14.4 Connection Security Checklist

Control	Guidance
Never store in .py files	Use Variable.get() or BaseHook.get_connection() instead of hardcoded strings.
Use read-only DB users	Create a dedicated airflow_read_only DB user with SELECT only privileges.
Rotate credentials	Update Airflow connections when passwords rotate. Use Secrets Manager for auto-rotation.
TLS/SSL everywhere	Set conn_extra: {"sslmode": "require"} for all PostgreSQL connections.
Audit connection access	Enable AIRFLOW__WEBSERVER__AUDIT_VIEW_DASBOARD_STATS to log connection reads.
Encrypt Fernet key	Set AIRFLOW__CORE__FERNET_KEY to a strong random key. Rotate annually.

Step 4 — Generate and set a new Fernet key

```
# Generate a Fernet encryption key for the metadata DB
$ python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"

# Set in .env
AIRFLOW__CORE__FERNET_KEY=<paste-generated-key-here>

# Restart for the key to take effect
$ astro dev restart
```

15 IMPLEMENT RBAC ROLES

Airflow 2.x ships with a built-in **Role-Based Access Control (RBAC)** system. Access to DAGs, connections, variables, and UI views can be restricted by role. Astronomer Cloud extends this with workspace-level and deployment-level roles managed through the Astronomer UI or CLI.

15.1 Built-In Airflow Roles

Role	Permissions
Admin	Full access to all DAGs, connections, variables, users, and configuration. Assign sparingly.
Op (Operator)	Can trigger DAGs, clear task instances, edit variables. Cannot manage users or connections.
User	Can view DAGs, trigger runs, and view logs. Cannot edit connections or variables.
Viewer	Read-only: view DAGs, task logs, and run history. No write actions.
Public	No permissions. Used for unauthenticated access (effectively blocks all actions).

15.2 Create and Manage Users via the CLI

Step 1 — Create users with specific roles

```
# Create a Viewer user
$ astro dev run users create \
--role Viewer \
--username data_analyst \
--firstname Priya \
--lastname Sharma \
--email priya@example.com \
--password 'TempPass@123'

# Create an Operator user
$ astro dev run users create \
--role Op \
--username pipeline_dev \
--email dev@example.com \
--password 'DevPass@456'

# List all users
$ astro dev run users list

# Delete a user
$ astro dev run users delete --username data_analyst
```

15.3 Create Custom Roles via the Airflow UI

Step 2 — Navigate to Security → List Roles → + Add

Airflow permissions follow the pattern **can_<action> on <resource>**. You can combine fine-grained permissions to build tailored roles. Common permissions:

Permission	Effect
can_read on DAGs	View DAG list, graph, code, and task status.

can_edit on DAGs	Toggle pause/unpause, trigger, and clear runs.
can_read on Task Instances	View task logs and instance details.
can_read on Variables	View variable keys (values are masked by default).
can_edit on Variables	Create, update, delete variables.
can_read on Connections	View connection IDs (passwords always masked).
can_edit on Connections	Create, update, delete connections.
menu access on Admin	Show or hide the Admin menu entry.

15.4 DAG-Level Access Control

Step 3 — Restrict a DAG to specific roles using access_control

```
@dag(
dag_id="finance_etl",
schedule="@daily",
access_control={
"Finance_Team": {"can_read", "can_edit"},
"Data_Auditor": {"can_read"},
},
)
def finance_etl():
...
```

✦ **NOTE** DAG-level access_control only works when AIRFLOW__WEBSERVER__RBAC=True (default in Airflow 2.x). Users with the Admin role always bypass DAG-level restrictions.

15.5 Astronomer Cloud RBAC

Astronomer Role	Scope
Workspace Admin	Full control over all deployments, members, and billing in the workspace.
Workspace Author	Can create and deploy to deployments. Cannot manage members or workspace settings.
Workspace Operator	Can trigger DAGs, view logs, manage connections on existing deployments. Cannot deploy code.
Workspace Viewer	Read-only access to deployment status and DAG run history. Cannot trigger or modify.

Step 4 — Manage workspace members via CLI

```
# List workspace users
$ astro workspace user list

# Add a user with Operator role
$ astro workspace user add priya@example.com \
--role WORKSPACE_OPERATOR

# Update a user's role
$ astro workspace user update priya@example.com \
--role WORKSPACE_VIEWER

# Remove a user from the workspace
$ astro workspace user remove priya@example.com
```

15.6 RBAC Hardening Checklist

Control	Action
Disable Public role	Remove all permissions from the Public role to block unauthenticated access.
Change admin password	Immediately change the default admin/admin password in any non-local environment.
Enable LDAP/SAML	Integrate with corporate IdP via AUTH_BACKEND=airflow.contrib.auth.backends.ldap_auth.
Audit role assignments	Review Security → List Users monthly. Remove stale accounts promptly.
Principle of least privilege	Assign the Viewer role by default. Elevate to Op/Admin only when required.
Session timeout	Set AIRFLOW__WEBSERVER__WEB_SERVER_SSL_CERT and SESSION_LIFETIME_DAYS=1.

Lab Completion Summary

Section 9	**Deploy DAG to Astro Cloud**	astro login → astro deploy --dags → CI/CD with GitHub Actions
Section 10	**Trigger DAG Runs**	UI trigger · CLI trigger · REST API · Dataset-based triggers
Section 11	**Inspect Logs**	UI log viewer · astro dev logs · /usr/local/airflow/logs/ · remote S3
Section 12	**Debug Deployment Errors**	build logs · dags list-import-errors · astro dev pytest · dag.test()
Section 13	**Configure Secrets**	.env · AWS Secrets Manager · Variable.get() · Fernet encryption
Section 14	**Setup Secure Connections**	UI / CLI connections · TLS · Fernet · read-only DB users
Section 15	**Implement RBAC Roles**	Built-in roles · custom roles · DAG access_control · Astronomer Cloud RBAC

✔ **TIP** Recommended path forward: (1) Add Slack/email alerts via apache-airflow-providers-slack. (2) Build an SLA-aware pipeline using sla_miss_callback. (3) Explore Airflow 2.7+ Dynamic Task Mapping for variable-fan-out pipelines. (4) Set up Prometheus + Grafana monitoring for your Astro deployment.

End of Lab Guide — Apache Airflow with Astro CLI (Continuation Sections 9–15)